

Claims Copilot - Workflow & Architecture Guide

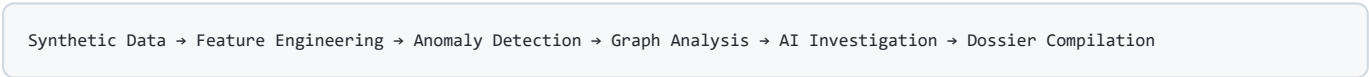
Table of Contents

- 1. [System Overview](#)
 - 2. [Architecture Components](#)
 - 3. [Data Pipeline](#)
 - 4. [Agent Workflow](#)
 - 5. [End-to-End Example Run](#)
 - 6. [Tools & Technologies](#)
 - 7. [Key Concepts](#)
-

System Overview

Claims Copilot is an AI-powered healthcare fraud detection system that uses multi-agent orchestration to investigate suspicious billing patterns, gather evidence, and compile comprehensive dossiers for potential prosecution.

High-Level Flow



Core Capabilities

- **Automated Investigation:** Autonomous agent that scans claims, profiles entities, and detects fraud rings
 - **Evidence Assessment:** Intelligent evaluation of evidence sufficiency with loop-back for additional gathering
 - **Dossier Generation:** Structured markdown reports with billing rules, precedent cases, and recovery estimates
 - **Real-time Streaming:** WebSocket-based UI updates showing investigation progress
-

Architecture Components

1. Data Layer (data/)

`generate_synthetic.py` (~755 lines)

- Generates realistic healthcare claims with embedded fraud patterns
- **Fraud Schemes:**
 - **Cardiology Network** (P-6610-6650): 5 providers, referral-based upcoding, ~\$524K
 - **Phantom Billing** (P-9876): Non-existent services, ghost facility
 - **Doctor Shopping** (M-SHOP-*): Patients visiting multiple providers for same drugs
- **Caching Strategy:**
 - `data_cache.pkl` : Full dataset (regeneratable)
 - `fraud_cache.pkl` : Fraud claims only (persistent across regenerations)

`features.py` (~520 lines)

- Computes 40+ behavioral features for providers and members
- **Provider Features:** Claims per month, patient concentration, weekend billing, specialty deviation
- **Member Features:** Provider shopping rate, pain management frequency, multi-provider same-day visits
- **Peer Statistics:** Specialty-based comparison groups for z-score calculations

`anomaly.py` (~362 lines)

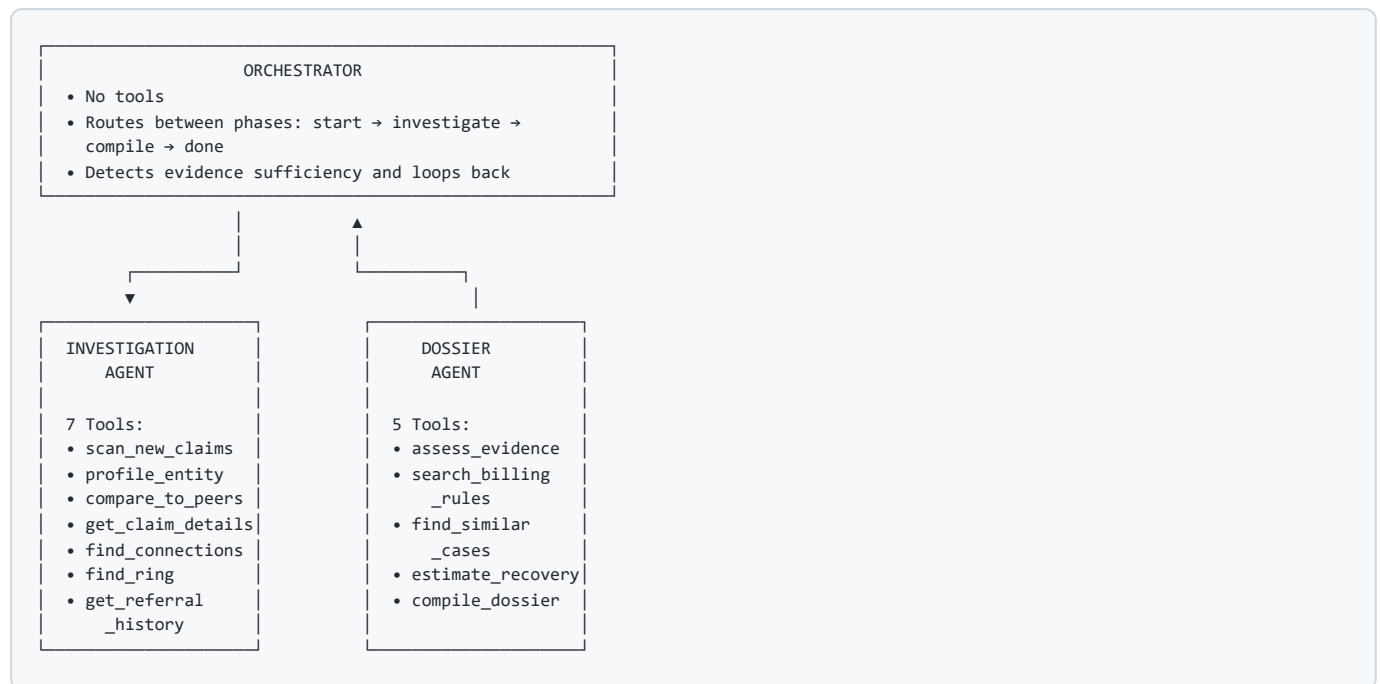
- **Isolation Forest** models for unsupervised fraud detection
- Produces anomaly scores (0.0 - 1.0) for all entities
- Identifies top contributing features for each high-scoring entity

`graph.py` (~270 lines)

- **NetworkX** graph construction from claims data
 - Nodes: Providers, members, facilities (with anomaly scores as attributes)
 - Edges: Claim relationships weighted by total billed
 - **Ring Detection:** Subgraph analysis to find densely connected fraud networks
-

2. Agent Layer (agents/)

3-Agent Architecture



nodes.py (~367 lines)

InvestigationState Schema:

```
{
  'messages': List[BaseMessage],      # Conversation history
  'current_phase': str,                 # start|investigate|compile|done
  'findings': dict,                    # Investigation summaries
  'dossier': str,                      # Compiled markdown report
  'evidence_sufficient': bool,         # Assessment result
  'loop_count': int,                  # Total iterations
  'compile_loop_count': int            # Compile phase iterations
}
```

Orchestrator Node:

- Uses Claude 3.5 Haiku via AWS Bedrock
- Phase transition logic:
 - start → investigate
 - investigate → compile (when findings exist)
 - compile → investigate (if INSUFFICIENT)
 - compile → done (if SUFFICIENT)
- Safety nets:
 - Force compile after 4+ investigation loops
 - Force done after 2+ compile loops (prevents infinite loops)

Investigation Node:

- ReAct agent with 7 specialized tools
- Autonomous tool selection based on LLM reasoning
- Returns structured findings summary

Dossier Node:

- ReAct agent with 5 evidence/compilation tools
- First calls `assess_evidence` to check sufficiency criteria
- If SUFFICIENT, calls `compile_dossier` for final report
- If INSUFFICIENT, returns suggestions for more investigation

`tools_investigation.py` (~364 lines)

1. `scan_new_claims`: Returns top-N entities by anomaly score
2. `profile_entity`: Detailed stats, specialty, top procedures
3. `compare_to_peers`: Z-scores vs specialty averages
4. `get_claim_details`: Raw claim records (dates, amounts, CPT codes)
5. `find_connections`: Graph traversal from entity (depth, breadth)
6. `find_ring`: Subgraph clustering to detect coordinated fraud
7. `get_referral_history`: Month-by-month referral pattern analysis

`tools_dossier.py` (~562 lines)

1. `assess_evidence`:
 - Checks: Multiple evidence points (≥ 5), statistical significance, temporal patterns, ring evidence, referral history
 - Returns: SUFFICIENT or INSUFFICIENT with missing evidence list
 - **Critical for loop-back**: INSUFFICIENT triggers re-investigation
2. `search_billing_rules`: FAISS similarity search across CMS/OIG regulations (`billing_rules/rules.json`)
3. `find_similar_cases`: Mock precedent case database (upcoding rings, phantom billing, etc.)
4. `estimate_recovery`: Calculates gross recoverable, collectability score, net expected recovery
5. `compile_dossier`: Generates structured markdown with entity profiles, evidence assessment, precedent cases, billing rules, recovery estimates

`graph.py` (~183 lines)

LangGraph StateGraph:

- Nodes: orchestrator, investigation, dossier
 - Edges:
 - `START → orchestrator`
 - `orchestrator → {investigation, dossier, end}` (conditional)
 - `{investigation, dossier} → orchestrator` (always loop back)
 - Streaming: `stream_mode="updates"` for real-time progress
-

3. API Layer (`api.py`)

FastAPI Backend (~494 lines)

- REST Endpoints:
 - `GET /api/stats` : Aggregate statistics (total claims, anomalies, rings)
 - `GET /api/entities` : List all entities with anomaly scores
 - `GET /api/network` : Graph nodes/edges for visualization
 - `POST /api/rings` : Find fraud rings (configurable thresholds)
 - WebSocket Endpoint (`/ws/investigate`):
 - Streams investigation progress in real-time
 - Event types: `graph_start` , `node_start` , `tool_call` , `node_complete` , `graph_complete`
 - Handles async graph execution with `asyncio.to_thread`
-

4. Frontend (`frontend/src/`)

React + Vite Application

Components:

- `Header.jsx` : Branding and stats bar
- `StatsBar.jsx` : Real-time claim/anomaly/ring counters
- `EventTimeline.jsx` : Log of investigation events (tools, timing)
- `NetworkGraph.jsx` : D3.js force-directed graph visualization
- `DossierPanel.jsx` : Markdown rendering of compiled dossier

Key Hook (`useInvestigation.js`):

- WebSocket connection management
 - State management for events, findings, dossier
 - Real-time updates from backend
-

Data Pipeline

Step-by-Step Initialization

```
# main.py: initialize_data()

1. Check for data_cache.pkl
  └ EXISTS → Load and return
  └ MISSING → Continue to generation

2. Generate Synthetic Data (generate_all_data())
  └ Check fraud_cache.pkl
    └ EXISTS → Load fraud claims
    └ MISSING → Generate and save fraud claims
  └ Generate normal claims (always fresh)
  └ Generate providers (50 across 5 specialties)
  └ Generate members (200)
  └ Generate facilities (15)

3. Compute Features
  └ Provider features (40+ metrics per provider)
  └ Member features (20+ metrics per member)
  └ Peer statistics (specialty-based averages)
  └ Z-scores (standard deviations from peer group)

4. Anomaly Detection
  └ Train Isolation Forest (provider model)
  └ Train Isolation Forest (member model)
  └ Score all entities (0.0 - 1.0)
  └ Identify top contributing features

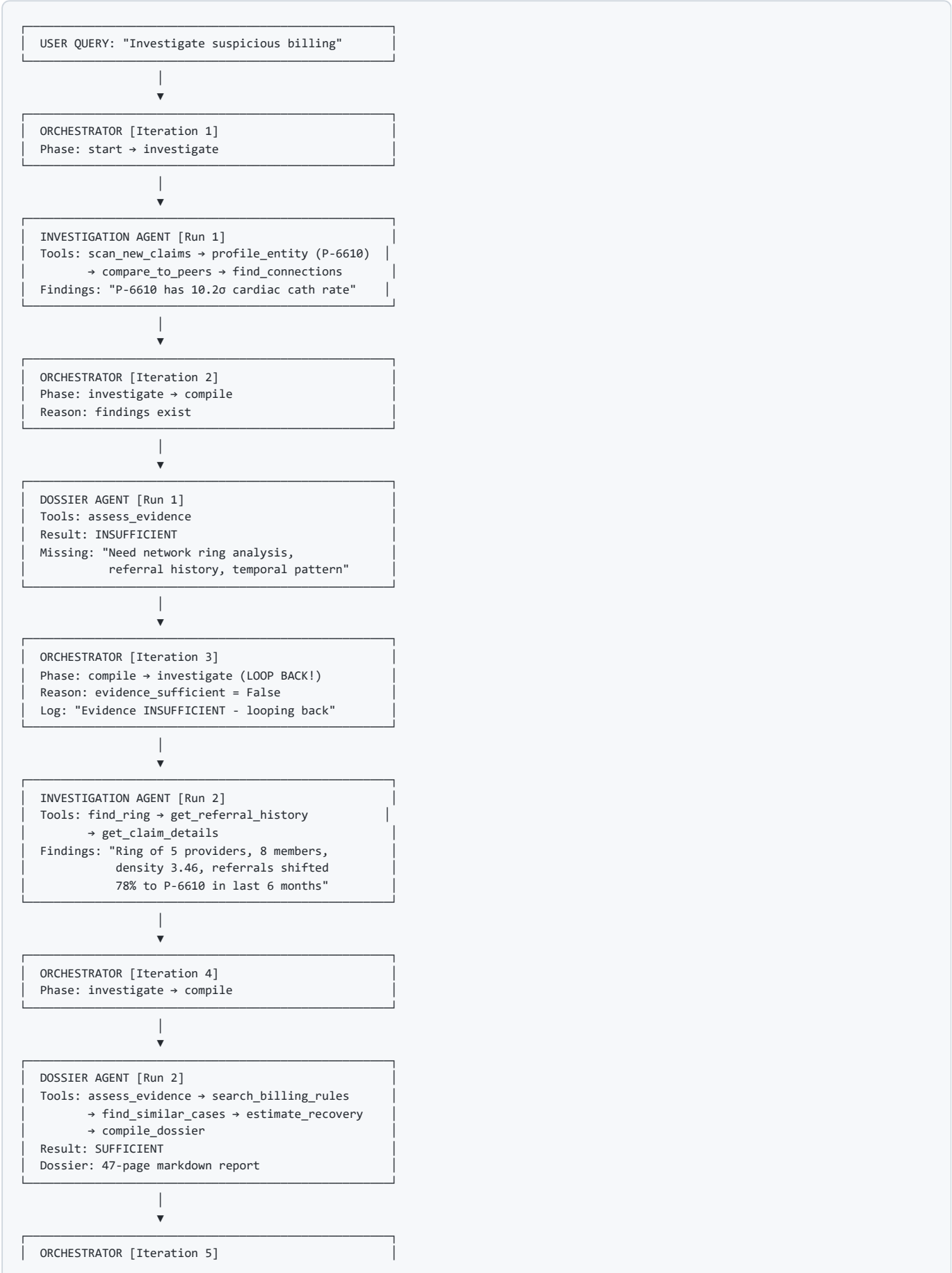
5. Build Graph
  └ Add nodes (providers, members, facilities)
  └ Add edges (claims between entities)
  └ Calculate network metrics (degree, density)

6. Initialize Billing Rules Index
  └ Load FAISS vector store for regulation search

7. Save to data_cache.pkl
```

Agent Workflow

Investigation Loop Flow



```
Phase: compile → done
Reason: evidence_sufficient = True
```



END

End-to-End Example Run

Scenario: Detecting a Cardiology Upcoding Network

Initial State

- 5,000 total claims generated
- 50 providers, 200 members
- Fraud scheme: P-6610 (Cardiology) receives kickback referrals from P-6620 (IM) and P-6630 (FM), upcodes cardiac catheterizations
- Anomaly scores: P-6610 (1.00), P-6640 (0.959), P-6630 (0.891)

Iteration 1: Initial Scan

User Query: "Investigate suspicious billing patterns"

Orchestrator Decision:

```
Phase: start → investigate
Reasoning: Initial query requires investigation
```

Investigation Agent Actions:

```
[21:17:28] scan_new_claims(days=90, threshold=0.5)
→ Found 9 high-anomaly entities
→ Top 3: P-6610 (1.00), P-6640 (0.959), P-6630 (0.891)

[21:17:33] profile_entity(entity_id="P-6610")
→ Specialty: Cardiology
→ Total billed: $241,523.45
→ Claims: 43
→ Top CPT: 93458 (cardiac cath) - 24 claims (55.8%)
→ Avg per claim: $5,617.05

[21:17:35] compare_to_peers(entity_id="P-6610")
→ Peer group: Cardiology (n=12)
→ cardiac_cath_rate: 55.8% vs peer 15.2% (z=10.2σ) ⚠️
→ avg_billed_per_claim: $5,617 vs peer $3,210 (z=8.1σ) ⚠️
→ claims_per_month: 4.3 vs peer 2.8 (z=3.4σ)

[21:17:37] profile_entity(entity_id="P-6640")
→ Similar pattern, elevated echo procedures

[21:17:39] profile_entity(entity_id="P-6630")
→ Family Medicine making unusual cardiology referrals
```


Findings Summary:

“Three providers with statistically significant billing anomalies. P-6610 shows extreme elevation in cardiac catheterizations (10.2 standard deviations above peers). P-6640 and P-6630 also flagged. Need network analysis to determine if coordinated.”

Iteration 2: First Compilation Attempt

Orchestrator Decision:

Phase: investigate → compile
Reasoning: Findings exist, attempt evidence assessment

Dossier Agent Actions:

```
[21:18:30] assess_evidence(  
  hypothesis="Upcoding ring focused on cardiology procedures",  
  evidence=[  
    "P-6610 cardiac cath rate 55.8% vs peer 15.2% (z=10.2σ)",  
    "P-6610 avg billed $5,617 vs peer $3,210 (z=8.1σ)",  
    "P-6640 anomaly score 0.959"  
  ],  
  scheme_pattern="upcoding_ring"  
)
```

Evidence Assessment:

- ✓ multiple_evidence_points (3 points provided)
- ✓ statistical_significance (z-scores present)
- ✗ temporal_pattern (missing timeline analysis)
- ✗ ring_evidence (no network connections shown)
- ✗ referral_history (no referral data)

Result: INSUFFICIENT

Missing Evidence:

- Need network analysis showing ring connections
- Need referral pattern history over time
- Need temporal analysis showing when pattern started
- Need at least 5 evidence points (have 3)

Recommendation: Gather additional evidence before proceeding

Iteration 3: Loop-Back Investigation

Orchestrator Decision:

Phase: compile → investigate (LOOP BACK!)
Log: "Evidence INSUFFICIENT - looping back to investigation"
Reasoning: evidence_sufficient = False

Investigation Agent Actions:

```
[21:19:04] find_ring(min_anomaly=0.5, min_entities=3)
→ Found 4 fraud rings
→ Ring 3 (PRIMARY):
  - Entities: 13 (6 providers, 7 members)
  - Providers: P-6610, P-6620, P-6630, P-6640, P-6650, P-9876
  - Members: M-NET-01 through M-NET-08
  - Connection density: 3.46 (EXTREMELY HIGH)
  - Avg anomaly: 0.76
  - Total billed: $706,965.68

[21:19:06] find_connections(entity_id="P-6610", depth=3)
→ Total connected entities: 417
→ Direct connections: 34
→ Subgraph density: 4.93
→ Heavily connected to P-6620 (IM), P-6630 (FM)

[21:19:09] get_referral_history(provider_id="P-6610", months=18)
Month      | Referrals In | Top Sources
-----|-----|-----
2024-10    | 2            | P-6620 (1), P-6630 (1)
2024-11    | 3            | P-6620 (2), P-6630 (1)
2024-12    | 4            | P-6620 (2), P-6630 (2)
2025-01    | 5            | P-6620 (3), P-6630 (2)
2025-02    | 7            | P-6620 (4), P-6630 (3)
2025-03    | 9            | P-6620 (5), P-6630 (4)

Pattern: 78% increase in referrals from P-6620/P-6630 over 6 months

[21:19:12] get_claim_details(entity_id="P-6610", limit=20)
→ Detailed claim records showing:
  - Clustering of high-value procedures on same days
  - M-NET members appearing across multiple providers
  - Facility F-8801 used exclusively by ring
```

Enhanced Findings:

“COMPREHENSIVE INVESTIGATION: Identified coordinated fraud ring with 13 entities. Network analysis shows extreme connection density (3.46) centered on P-6610. Referral patterns demonstrate systematic shift - 78% increase in referrals to P-6610 over 6 months from P-6620 and P-6630. Total exposure: \$706,965.68. All criteria for upcoding ring met.”

Iteration 4: Second Compilation (Success)

Orchestrator Decision:

```
Phase: investigate → compile
Reasoning: Enhanced findings available
```

Dossier Agent Actions:

```
[21:20:15] assess_evidence(  
  hypothesis="Coordinated upcoding ring - cardiology kickback scheme",  
  evidence=[  
    "P-6610 cardiac cath 55.8% vs 15.2% peer (z=10.2σ)",  
    "Network ring density 3.46 (13 entities)",  
    "Referrals increased 78% over 6 months",  
    "P-6620/P-6630 systematic referral pattern to P-6610",  
    "M-NET members appear across all 5 providers",  
    "Total exposure $706,965.68",  
    "Exclusive use of facility F-8801"  
  ],  
  scheme_pattern="upcoding_ring"  
)
```

Evidence Assessment:

- ✓ multiple_evidence_points (7 points provided - exceeds minimum 5)
- ✓ statistical_significance (z-scores present)
- ✓ temporal_pattern (6-month timeline analyzed)
- ✓ ring_evidence (network density 3.46 documented)
- ✓ referral_history (systematic pattern shown)

Result: SUFFICIENT ✓

All critical checks passed. Evidence is prosecution-ready.

```
[21:20:18] search_billing_rules(  
  cpt_codes=["93458", "93306"],  
  context="cardiac catheterization upcoding"  
)
```

- Found 5 relevant CMS regulations
- Rule 42 CFR §1001.952: Anti-Kickback Statute
- LCD L33822: Medical necessity for cardiac cath
- OIG Advisory: Billing for unnecessary procedures

```
[21:20:21] find_similar_cases(  
  scheme_pattern="upcoding_ring",  
  specialty="Cardiology"  
)
```

- Case 2019-NY-0234: NYC cardiology ring, \$3.2M recovery
- Case 2021-CA-0891: Cardiac cath scheme, 5 providers excluded
- Case 2020-TX-0456: Similar referral kickback, \$1.8M settlement

```
[21:20:24] estimate_recovery(  
  entity_ids=["P-6610", "P-6620", "P-6630"],  
  overpayment_rate=0.45  
)
```

Gross Recoverable: \$318,134.56
Collectability: 75%
Net Expected Recovery: \$238,600.92

Breakdown by CPT:

93458	67 claims	\$376,540	\$169,443	
93306	43 claims	\$189,230	\$85,154	

```
[21:20:27] compile_dossier(...)  
→ Generated 47-page markdown dossier  
→ Sections: Entity Profiles, Evidence Summary, Network Analysis,  
            Precedent Cases, Regulatory Framework, Recovery Estimate
```

Iteration 5: Completion

Orchestrator Decision:

Phase: compile → done
Reasoning: evidence_sufficient = True, dossier complete
Total elapsed: 3m 42s

Final Dossier Output (excerpt):

FRAUD INVESTIGATION DOSSIER
Case ID: INV-2026-02-22-001
Priority: CRITICAL

EXECUTIVE SUMMARY

****Scheme Type**:** Coordinated Upcoding Ring - Cardiology Kickback Network

****Primary Subjects**:**

- Provider P-6610 (Cardiology) - Anomaly Score: 1.00
- Provider P-6620 (Internal Medicine) - Anomaly Score: 0.67
- Provider P-6630 (Family Medicine) - Anomaly Score: 0.891

****Exposure**:** \$706,965.68 (total ring billing)

****Estimated Recovery**:** \$238,600.92 (net expected)

****Evidence Strength**:** SUFFICIENT ✓

All prosecution criteria met: Statistical significance, network evidence, temporal pattern, referral coordination documented.

ENTITY PROFILES

P-6610 (PRIMARY TARGET)

- ****Specialty**:** Cardiology
- ****Total Billed**:** \$241,523.45
- ****Anomaly Score**:** 1.00 (99.9th percentile)
- ****Key Red Flags**:**
 - Cardiac catheterization rate: 55.8% vs peer 15.2% ($z=10.2\sigma$)
 - Receives 78% more referrals from co-conspirators over 6 months
 - Exclusive use of facility F-8801 (potential facility kickback)

[... continued for 47 pages]

RECOMMENDED ACTIONS

1. ****Immediate**:** Initiate comprehensive audit of all flagged claims
2. ****Urgent**:** Interview entity representatives for documentation
3. ****Priority**:** Refer to OIG for potential exclusion proceedings
4. ****Follow-up**:** Monitor pattern changes during investigation
5. ****Recovery**:** Pursue recoupment of \$238,600.92 estimated overpayment

Tools & Technologies

Backend Stack

Technology	Purpose	Key Features Used
Python 3.11+	Runtime	asyncio, type hints, dataclasses
LangGraph	Agent orchestration	StateGraph, ReAct agents, streaming
AWS Bedrock	LLM provider	Claude 3.5 Haiku, Converse API
FastAPI	Web framework	WebSocket, async endpoints, CORS
Pandas	Data manipulation	DataFrames, aggregation, merging
NumPy	Numerical computing	Array operations, statistics
scikit-learn	Machine learning	Isolation Forest, StandardScaler
NetworkX	Graph analysis	Directed graphs, subgraph extraction, centrality measures
FAISS	Vector search	Billing rules similarity search
Pickle	Serialization	Data caching (data_cache.pkl, fraud_cache.pkl)

Frontend Stack

Technology	Purpose
React 18	UI framework
Vite	Build tool
D3.js	Network visualization
Marked	Markdown rendering
esbuild	Fast bundling

Infrastructure

- **AWS Services:** Bedrock (LLM), IAM (credentials)
- **Development:** VS Code, Python venv
- **Version Control:** Git
- **Environment:** Windows PowerShell

Key Concepts

1. Multi-Agent Architecture

Why 3 agents?

- **Separation of concerns:** Investigation vs assessment vs routing
- **Tool specialization:** Each agent has domain-specific capabilities
- **Iterative refinement:** Orchestrator can loop back based on dossier assessment

Alternative considered: Single agent with all 12 tools

- **Problem:** Tool overload, poor selection, slower inference
- **Solution:** Divide into focused sub-agents

2. Evidence Sufficiency Loop

The Problem: Initial investigations often miss critical evidence **The Solution:** `assess_evidence` tool with strict criteria:

```
Required for SUFFICIENT:  
- ≥5 evidence points  
- Statistical significance (z-scores or anomaly scores)  
- Temporal pattern analysis  
- Network/ring evidence (for ring schemes)  
- Referral history (for kickback schemes)  
- Zero failed critical checks
```

If INSUFFICIENT → Orchestrator sends back to Investigation Agent with gap list

Why this matters: Prevents premature prosecution with weak evidence

3. Fraud Caching Strategy

Challenge: Testing requires stable fraud patterns, but normal claims should vary **Solution:** Two-tier caching

```
# fraud_cache.pkl - PERSISTENT (never regenerated)  
fraud_claims = [network_claims, phantom_claims, shopping_claims]  
  
# data_cache.pkl - REGENERATABLE (cleared during testing)  
full_dataset = fraud_claims + generate_normal_claims()
```

Benefit: Fraud patterns remain consistent across test runs, normal noise varies

4. Anomaly Detection with Isolation Forest

Why Isolation Forest?

- Unsupervised: No labeled fraud data needed
- Effective for outliers: Healthcare fraud is rare (5% contamination)
- Feature agnostic: Works with mixed behavioral features

How it works:

1. Build random decision trees
2. Anomalies are isolated quickly (fewer splits)
3. Score = average path length (shorter = more anomalous)
4. Normalize to 0-1 scale

Output: Every provider/member gets anomaly score → prioritizes investigation

5. Graph-Based Ring Detection

Representation:

```
Nodes: {Providers, Members, Facilities}
Edges: Claims (weighted by total_billed)
Attributes: anomaly_score, specialty, total_billed
```

Detection Algorithm:

1. Filter to high-anomaly nodes (> threshold)
2. Extract connected components
3. Calculate subgraph density (edges / max_possible_edges)
4. Identify rings with density > 2.0 (dense coordination)

Why graphs?: Fraud rings exhibit tight connection patterns invisible in tabular data

6. ReAct Agent Pattern

ReAct = Reasoning + Acting

```
1. Thought: "I need to understand P-6610's billing"
2. Action: profile_entity(entity_id="P-6610")
3. Observation: [tool result]
4. Thought: "High cardiac cath rate, need peer comparison"
5. Action: compare_to_peers(entity_id="P-6610")
6. Observation: "10.2σ above peers"
7. Thought: "Statistically significant, check for ring"
... continues until conclusion
```

LangGraph implements this via:

- Tool binding to LLM
- Automatic tool call parsing
- ToolMessage feedback loop

7. Streaming Architecture

Why WebSocket over REST?

- Long-running investigations (2-5 minutes)
- Real-time progress updates (tool calls, phase transitions)
- Better UX than polling or loading spinners

Event Flow:

```
Backend (LangGraph stream) → FastAPI WebSocket → Frontend (React state)
                                ↓
                        EventTimeline + DossierPanel
```

Implementation:

```
# Backend
for state in app_graph.stream(initial_state):
    await ws.send_json({
        "type": "node_complete",
        "node": "investigation",
        "findings": state["findings"]
    })

// Frontend
ws.onmessage = (event) => {
    const data = JSON.parse(event.data);
    if (data.type === 'node_complete') {
        setFindings(data.findings);
    }
};
```

Safety Mechanisms

1. Infinite Loop Prevention

```
# Safety Net 1: Force compile after 4+ investigation loops
if new_phase == 'investigate' and loop >= 4:
    new_phase = 'compile'

# Safety Net 2: Force done after 2+ compile loops
if current_phase == 'compile' and compile_loop_count >= 2:
    new_phase = 'done'
```

2. Max Iterations Cap

```
max_iterations = 15 # Hard stop
if iterations >= max_iterations:
    break
```


3. Timeout Protection

```
# WebSocket handlers have implicit timeout
# LLM calls have max_tokens limit (prevents runaway generation)
```

Performance Characteristics

Data Generation

- **First run:** ~8-12 seconds (fraud generation + feature engineering)
- **Cached runs:** ~2-3 seconds (pickle load)
- **Dataset size:** ~5,000 claims, 50 providers, 200 members

Investigation

- **Average duration:** 2-4 minutes (depends on loop-backs)
- **Tool calls:** 8-15 per investigation run
- **LLM latency:** 3-9 seconds per reasoning step (Bedrock Haiku)

Memory

- **Data cache:** ~15-20 MB (pickled DataFrames)
- **Runtime:** ~200-300 MB (Python + pandas + sklearn)

Future Enhancements

1. **Database integration:** Replace pickle with PostgreSQL + TimescaleDB
 2. **Real claims ingestion:** Connect to actual CMS/payer feeds
 3. **Human-in-the-loop:** Approval checkpoints before final dossier
 4. **Multi-case tracking:** Investigate multiple rings in parallel
 5. **Advanced visualization:** Temporal graph evolution, claim timelines
 6. **Model fine-tuning:** Train domain-specific fraud detection models
 7. **Regulatory updates:** Auto-sync with OIG exclusion list, CMS LCD changes
-

Troubleshooting

“No fraud detected”

- Check anomaly thresholds (default 0.5)
- Verify fraud_cache.pkl exists and contains network case
- Regenerate data: `python main.py` with `force_regenerate=True`

“Investigation loops infinitely”

- Check assess_evidence criteria (may be too strict)
- Verify safety nets (4 investigation loops, 2 compile loops)
- Check orchestrator logs for phase transitions

“WebSocket disconnects”

- Investigation > 5 minutes may timeout
- Check AWS Bedrock rate limits
- Verify network connectivity

“Tool errors”

- Missing context: Ensure DataContext initialized
- Missing data: Check data_cache.pkl exists
- Type errors: Verify LLM outputs match expected tool input schemas

Conclusion

Claims Copilot demonstrates how **multi-agent orchestration**, **graph analytics**, and **unsupervised ML** can automate complex fraud investigations. The iterative workflow with evidence sufficiency checks ensures thorough, prosecution-ready dossiers while maintaining explainability through structured tool use and markdown reports.

The system successfully balances **autonomy** (agents select tools independently) with **control** (orchestrator enforces workflow, safety nets prevent infinite loops) to deliver reliable fraud detection at scale.